

Monte Carlo RTP Simulator

Paralelna simulacija slot mašina sa Intel TBB

Ispitni rad – Paralelno Programiranje

Kandidat: Nikola Pešić

Indeks: SV39/2024

Mentor: dr Dušan Kenjić

Novi Sad, juni 2026.

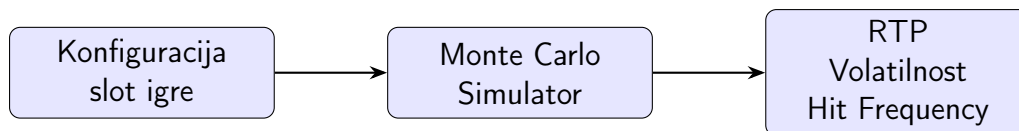
Sadržaj

1	Uvod	1
1.1	Opis problema	1
1.2	Cilj projekta	1
1.3	Tehnologije	1
2	Specifikacija	2
2.1	Funkcionalni zahtevi	2
2.2	Programski zahtevi	2
3	Arhitektura sistema	3
3.1	Slot Engine kao Git submodule	3
3.2	Struktura direktorijuma	3
3.3	Arhitektura Monte Carlo simulatora	4
4	Opis rešenja	4
4.1	Slot Engine integracija	4
4.2	Modul statistike (SimulationStatistics)	5
4.3	Modul Monte Carlo simulatora (MonteCarloSimulator)	5
4.4	Modul RTP Calculator (RTPCalculator)	6
5	Paralelizacija	6
5.1	parallel_reduce	6
5.2	enumerable_thread_specific	7
5.3	flow::graph	8
6	Rezultati	10
6.1	Primer izveštaja simulacije	10
6.2	Benchmark rezultati	11
6.3	Analiza rezultata	13
7	Testiranje	13
7.1	Strategija testiranja	13
7.2	Testni scenariji	13
8	Zaključak	14

1 Uvod

1.1 Opis problema

Pre nego što slot igra bude puštena na tržište, mora proći sertifikaciju koja verifikuje tačan **RTP (Return to Player)** – procenat uloga koji igra statistički vraća igraču. Da bi RTP bio statistički pouzdan, potrebno je simulirati između 100 i 500 miliona spinova. Sekvencijalno, to traje mnogo dugo.



Slika 1: Konceptualni dijagram Monte Carlo simulacije

1.2 Cilj projekta

Cilj ovog projekta je implementacija paralelnog Monte Carlo simulatora za slot mašine koji:

- Prima definiciju slot igre (reelovi, payline-ovi, paytable)
- Paralelno izračunava RTP, volatilnost i hit frequency
- Koristi Intel TBB biblioteku za paralelizaciju
- Demonstrira `parallel_reduce`, `enumerable_thread_specific` i `flow::graph`

1.3 Tehnologije

- **C++17** – jezik implementacije
- **Intel TBB** – biblioteka za paralelno programiranje
 - `parallel_reduce` – agregacija rezultata spinova
 - `enumerable_thread_specific` – thread-local RNG
 - `flow::graph` – modelovanje zavisnosti faza analize
- **CMake** – sistem za build
- **Git submodule** – Slot Engine kao eksterna komponenta

2 Specifikacija

2.1 Funkcionalni zahtevi

Tabela 1: Pregled funkcionalnih zahteva

ID	Zahtev	Opis
FR1	Učitavanje konfiguracije	Program učitava definiciju slot igre (reelovi, payline-ovi, paytable)
FR2	Simulacija spinova	Izvršava zadati broj spinova i beleži rezultate
FR3	Paralelizacija	Koristi TBB za paralelnu simulaciju spinova
FR4	RTP po komponentama	Računa RTP za base, bonus i scatter komponente
FR5	Volatilnost	Računa varijansu i standardnu devijaciju distribucije dobitaka
FR6	Hit frequency	Računa procenat spinova sa bilo kakvim dobitkom
FR7	Benchmark	Meri vreme izvršenja za 2, 4, 8 i 16 niti
FR8	Izveštaj	Generiše formatirani izveštaj sa svim statistikama
FR9	RTP validacija	Upoređuje simulirani RTP sa očekivanom vrednošću

2.2 Programski zahtevi

Tabela 2: Pregled programskih zahteva

ID	Zahtev	Opis
PR1	Modularna struktura	Svaka klasa ima zasebne .h i .cpp datoteke
PR2	Namespace	Svi simboli su u <code>namespace SlotEngine</code>
PR3	Thread-safety	SpinEngine je thread-safe, SlotEngine nije
PR4	Konzistentno imenovanje	Korišćenje konvencije imenovanja
PR5	Dokumentacija	Komentarisanje interfejsi klasa

3 Arhitektura sistema

3.1 Slot Engine kao Git submodule

Slot Engine je implementiran kao zasebna biblioteka i uključen u projekat kao **Git submodule**. Ovo omogućava:

- Nezavisan razvoj i verzionisanje Slot Engine-a
- Ponovnu upotrebu u drugim projektima
- Čistu separaciju odgovornosti – Monte Carlo simulator se bavi samo simulacijom, a ne logikom slot mašine

```
1 # Kloniranje sa submodulima
2 git clone --recursive https://github.com/Dzekanaa/monte-carlo-simulator.
  git
3
4 # A uriranje submodula
5 git submodule update --init --recursive
```

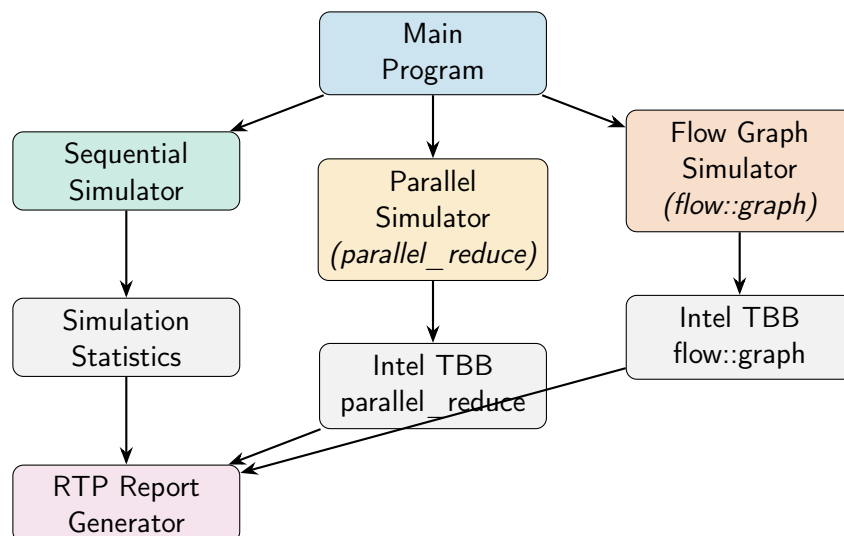
Listing 1: Uključivanje Slot Engine submodula

3.2 Struktura direktorijuma

```
1 monte-carlo-simulator/
2     CMakeLists.txt
3     .gitignore
4     .gitmodules                                # Referenca na Slot Engine
5     submodule
6         include/
7             flow_graph_simulator.h
8             monte_carlo_simulator.h
9             rtp_calculator.h
10            statistics.h
11        src/
12            flow_graph_simulator.cpp
13            main.cpp
14            monte_carlo_simulator.cpp
15            rtp_calculator.cpp
16            statistics.cpp
17        slot-engine/                            # Git submodule (Slot Engine
18        biblioteka)
19            include/
20                slot_engine.h
21            src/
22                ...
23        tests/
24            test_simulator.cpp
25        data/
26            (rezultati simulacija)
```

Listing 2: Struktura direktorijuma Monte Carlo simulatora

3.3 Arhitektura Monte Carlo simulatora



Slika 2: Arhitektura Monte Carlo simulatora

4 Opis rešenja

4.1 Slot Engine integracija

Monte Carlo simulator koristi **Slot Engine** biblioteku za izvršavanje spinova. Slot Engine pruža sledeći interfejs:

```

1 namespace SlotEngine {
2     class ISlotEngine {
3     public:
4         virtual SpinResult Spin() = 0;
5         virtual const GameConfig& GetConfig() const = 0;
6         virtual void SetConfig(const GameConfig& config) = 0;
7         virtual void Reset() = 0;
8     };
9
10    std::unique_ptr<ISlotEngine> CreateSlotEngine(
11        const GameConfig& config = GameConfig()
12    );
13
14    struct SpinResult {
15        int totalWin = 0;
16        int baseGameWin = 0;
17        int scatterWin = 0;
18        int bonusWin = 0;
19        bool triggeredFreeSpins = false;
20        int freeSpinsAwarded = 0;
21        bool IsHit() const;
22    };
23 }

```

Listing 3: Slot Engine API (korišćen u simulatoru)

4.2 Modul statistike (SimulationStatistics)

```

1 struct SimulationStatistics {
2     // Accumulators
3     long long totalBet = 0;
4     long long totalWin = 0;
5     long long totalBaseWin = 0;
6     long long totalScatterWin = 0;
7     long long totalBonusWin = 0;
8     long long hitCount = 0;
9     int totalSpins = 0;
10
11     // For calculation of variance
12     std::vector<long long> allWins;
13
14     // Operator for parallel_reduce
15     SimulationStatistics& operator+=(const SimulationStatistics& other);
16
17     // Methods for calculation
18     double GetRTP() const;
19     double GetHitFrequency() const;
20     double GetVariance() const;
21     double GetStandardDeviation() const;
22     double GetBaseRTP() const;
23     double GetScatterRTP() const;
24     double GetBonusRTP() const;
25
26     void PrintReport() const;
27 };

```

Listing 4: SimulationStatistics struktura

4.3 Modul Monte Carlo simulatora (MonteCarloSimulator)

```

1 class MonteCarloSimulator {
2 public:
3     explicit MonteCarloSimulator(const SlotEngine::GameConfig& config,
4                                   int betPerSpin = 10);
5
6     // Three ways of simulation
7     SimulationStatistics RunSequential(int numSpins);
8     SimulationStatistics RunParallel(int numSpins);
9     SimulationStatistics RunFlowGraph(int numSpins);
10
11     // Benchmark
12     struct BenchmarkResult {
13         int numThreads;
14         double sequentialTime;
15         double parallelTime;
16         double flowGraphTime;
17         double speedupParallel;
18         double speedupFlowGraph;
19         SimulationStatistics stats;
20     };
21
22     BenchmarkResult Benchmark(int numSpins, int maxThreads = 16);

```

```

23 void PrintReport(const SimulationStatistics& stats) const;
24 };

```

Listing 5: MonteCarloSimulator API

4.4 Modul RTP Calculator (RTPCalculator)

```

1 class RTPCalculator {
2 public:
3     RTPCalculator(const SlotEngine::GameConfig& config);
4
5     double CalculateTheoreticalRTP() const;
6     double CalculateSimulatedRTP(const SimulationStatistics& stats)
7     const;
8     bool ValidateRTP(double rtp, double tolerance = 0.5) const;
9
10    struct ConfidenceInterval {
11        double lower;
12        double upper;
13        double confidenceLevel;
14    };
15
16    ConfidenceInterval CalculateConfidenceInterval(
17        const SimulationStatistics& stats,
18        double confidenceLevel = 0.95
19    ) const;
20 };

```

Listing 6: RTPCalculator API

5 Paralelizacija

5.1 parallel_reduce

`parallel_reduce` se koristi za paralelnu simulaciju spinova. Svaki spin je nezavisan, što ga čini idealnim za `parallel_reduce`.

```

1 SimulationStatistics MonteCarloSimulator::RunParallel(int numSpins) {
2     using namespace tbb;
3
4     // Thread-local engine pool
5     enumerable_thread_specific<std::unique_ptr<SlotEngine::ISlotEngine>>
6     enginePool;
7
8     SimulationStatistics totalStats = parallel_reduce(
9         blocked_range<int>(0, numSpins),
10        SimulationStatistics(), // identity
11        [&](const blocked_range<int>& range, SimulationStatistics
12        localStats) {
13            // Every Thread has its own engine
14            if (!enginePool.local()) {
15                enginePool.local() = SlotEngine::CreateSlotEngine(
16                m_config);
17            }
18            auto& engine = *enginePool.local();

```

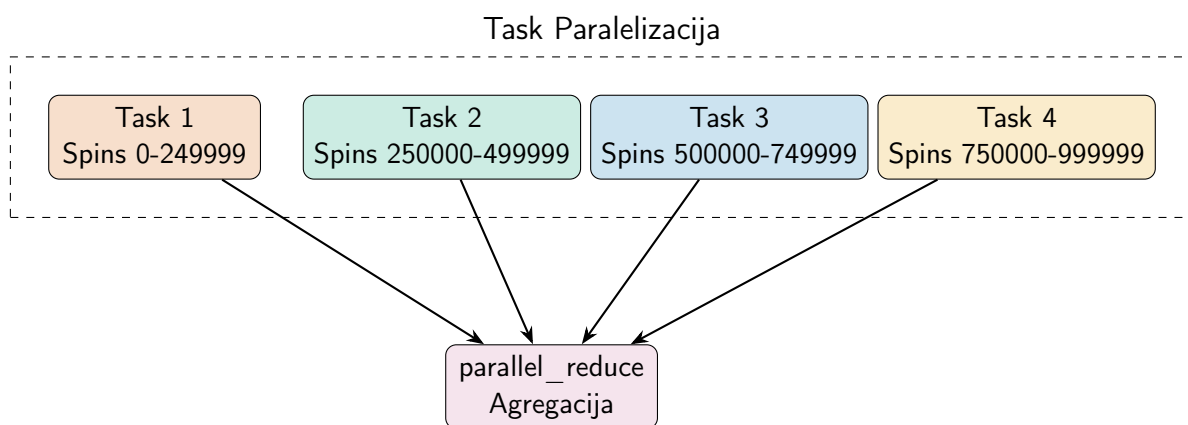


```

16
17     for (int i = range.begin(); i < range.end(); ++i) {
18         auto result = engine.Spin();
19         // Acumulation of local statistics
20         localStats.totalBet += m_betPerSpin;
21         localStats.totalWin += result.totalWin;
22         // ...
23     }
24     return localStats;
25 },
26 [](const SimulationStatistics& a, const SimulationStatistics& b)
27 {
28     return a + b; // agregation of local results
29 }
30 );
31 return totalStats;
32 }

```

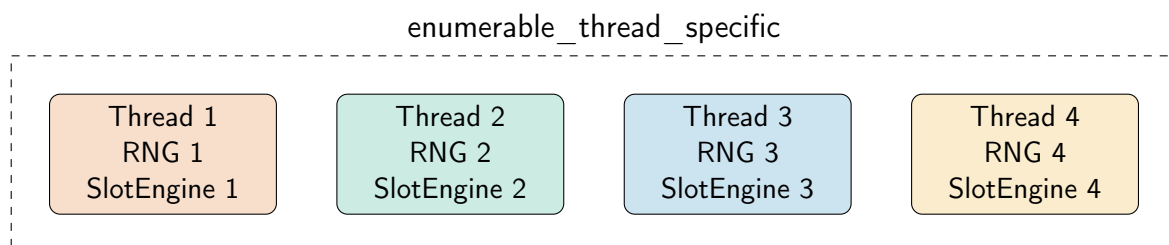
Listing 7: parallel_reduce implementacija



Slika 3: parallel_reduce model – paralelna simulacija spinova

5.2 enumerable_thread_specific

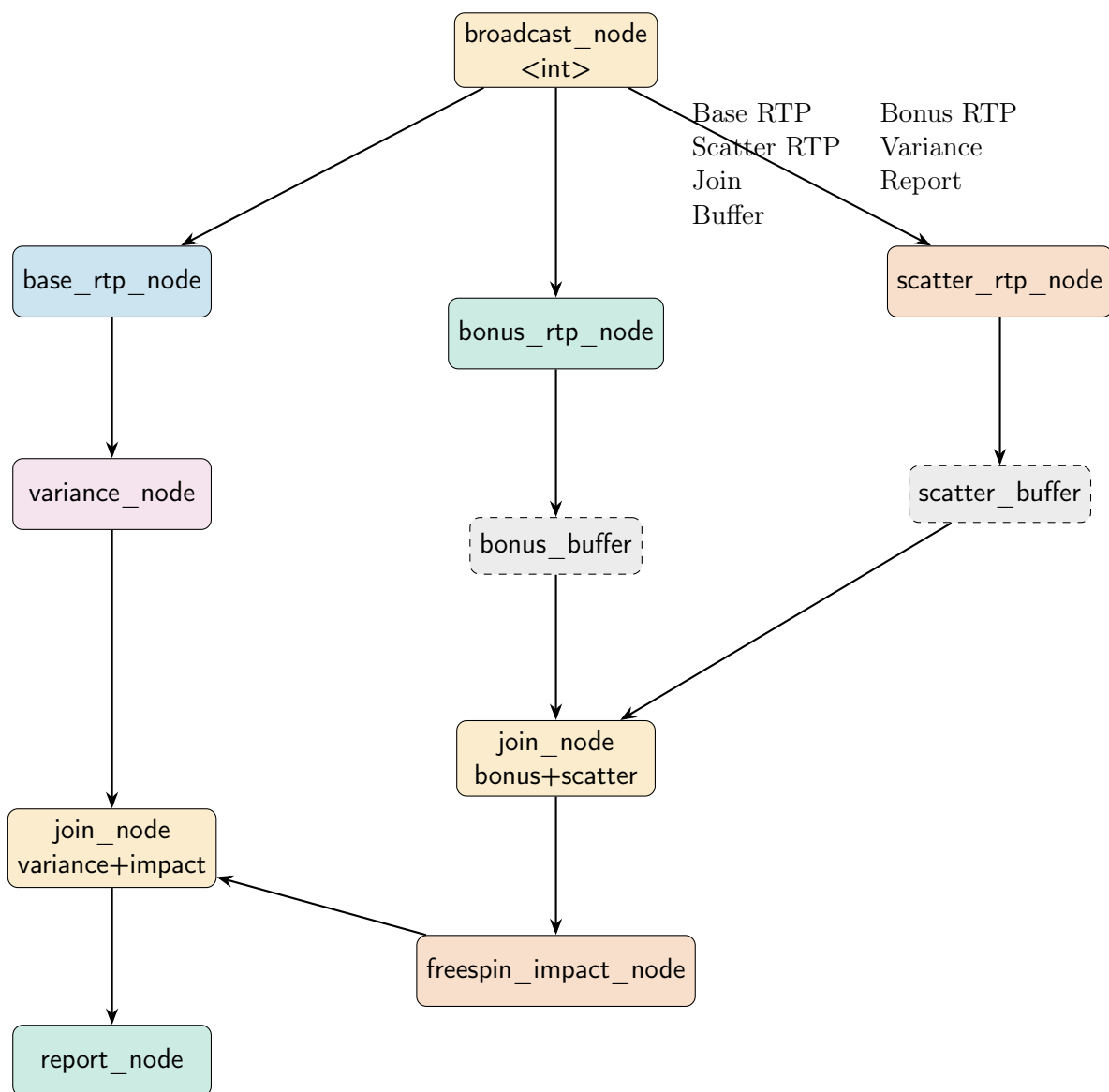
`enumerable_thread_specific` omogućava svakoj niti da ima sopstveni RNG i SlotEngine instancu, čime se eliminiše kontenzija oko deljenih resursa.



Slika 4: enumerable_thread_specific – svaka nit ima sopstveni RNG i SlotEngine

5.3 flow::graph

flow::graph se koristi za modelovanje zavisnosti između faza analize RTP-a. Graf omogućava prirodno izražavanje paralelizma i zavisnosti.



Slika 5: Struktura flow::graph-a sa buffer node-ovima

Opis čvorova:

Tabela 3: Čvorovi flow::graph-a

Čvor	Funkcija	Konkurentnost
broadcast_node	Šalje broj spinova svim komponentama	-
base_rtp_node	Simulira osnovnu komponentu igre	unlimited
bonus_rtp_node	Simulira bonus komponentu igre	unlimited
scatter_rtp_node	Simulira scatter komponentu igre	unlimited
bonus_buffer	Čuva bonus statistiku dok ne stigne scatter	buffer
scatter_buffer	Čuva scatter statistiku dok ne stigne bonus	buffer
join_node (bonus+scatter)	Spaja bonus i scatter rezultate	-
freespins_impact_node	Računa uticaj free spinova na RTP	serial
variance_node	Računa varijansu iz base dobitaka	serial
join_node (variance+impact)	Spaja variance i freespins impact	-
report_node	Generiše i ispisuje finalni izveštaj	serial

```

1 FlowGraphSimulator::FlowGraphSimulator(const Config& config, int bet)
2   : m_config(config)
3   , m_betPerSpin(bet)
4   , m_broadcast(m_graph)
5   , m_base_rtp_node(m_graph, unlimited, [this](int spins) {
6       return SimulateBaseComponent(spins);
7   })
8   , m_bonus_rtp_node(m_graph, unlimited, [this](int spins) {
9       return SimulateBonusComponent(spins);
10  })
11  , m_scatter_rtp_node(m_graph, unlimited, [this](int spins) {
12      return SimulateScatterComponent(spins);
13  })
14  , m_variance_node(m_graph, serial, [this](const SimulationStatistics
15      & baseStats) {
16      return ComputeVariance(baseStats);
17  })
18  , m_bonus_buffer(m_graph)
19  , m_scatter_buffer(m_graph)
20  , m_bonus_scatter_join(m_graph)
21  , m_freespins_impact_node(m_graph, serial, [this](const auto& data) {
22      return ComputeFreespinsImpact(data);
23  })
24  , m_final_join(m_graph)
25  , m_report_node(m_graph, serial, [this](const auto& data) {
26      GenerateReport(data);
27      return 0;
28  })
29 {
30     make_edge(m_broadcast, m_base_rtp_node);
31     make_edge(m_broadcast, m_bonus_rtp_node);
32     make_edge(m_broadcast, m_scatter_rtp_node);
33

```

```

34 // Base      Variance
35 make_edge(m_base_rtp_node, m_variance_node);
36
37 // Bonus      Buffer      Join (port 0)
38 make_edge(m_bonus_rtp_node, m_bonus_buffer);
39 make_edge(m_bonus_buffer, input_port<0>(m_bonus_scatter_join));
40
41 // Scatter      Buffer      Join (port 1)
42 make_edge(m_scatter_rtp_node, m_scatter_buffer);
43 make_edge(m_scatter_buffer, input_port<1>(m_bonus_scatter_join));
44
45 // Join      Freespin Impact
46 make_edge(m_bonus_scatter_join, m_freespin_impact_node);
47
48 // Variance      Final Join (port 0)
49 make_edge(m_variance_node, input_port<0>(m_final_join));
50
51 // Freespin Impact      Final Join (port 1)
52 make_edge(m_freespin_impact_node, input_port<1>(m_final_join));
53
54 // Final Join      Report
55 make_edge(m_final_join, m_report_node);
56 }

```

Listing 8: flow::graph implementacija sa buffer node-ovima

Zašto su buffer node-ovi potrebni?

join_node zahteva da sve ulazne poruke stignu istovremeno. U paralelnom sistemu, bonus_rtp_node i scatter_rtp_node mogu završiti u različito vreme. buffer_node rešava ovaj problem tako što čuva poruku dok ne stigne druga, pa tek onda šalje obe u join_node.

6 Rezultati

6.1 Primer izveštaja simulacije

```

1  MONTE CARLO RTP REPORT
2  -----
3  GENERAL STATISTICS:
4      Total spins:      1000000
5      Total bet:        1000000 coins
6      Total win:        972859 coins
7      Net result:       -27141 coins
8
9  RTP BREAKDOWN:
10     Total RTP:        97%
11     Base game RTP:    80.91%
12     Scatter RTP:      2.33%
13     Bonus RTP:        14.04%
14
15  VOLATILITY:
16     Hit frequency:    11.24%
17     Variance:         40.09
18     Std deviation:    6.33
19
20  WIN DISTRIBUTION (top 10):

```

```

21 # 1: 724 coins
22 # 2: 700 coins
23 # 3: 630 coins
24 # 4: 532 coins
25 # 5: 532 coins
26 # 6: 521 coins
27 # 7: 502 coins
28 # 8: 494 coins
29 # 9: 493 coins
30 #10: 490 coins
31
32 RTP Validation:
33   Expected RTP:      96.50%
34   Simulated RTP:     97%
35   Status:           VALID
36   95% Confidence:    [0.96, 0.99]

```

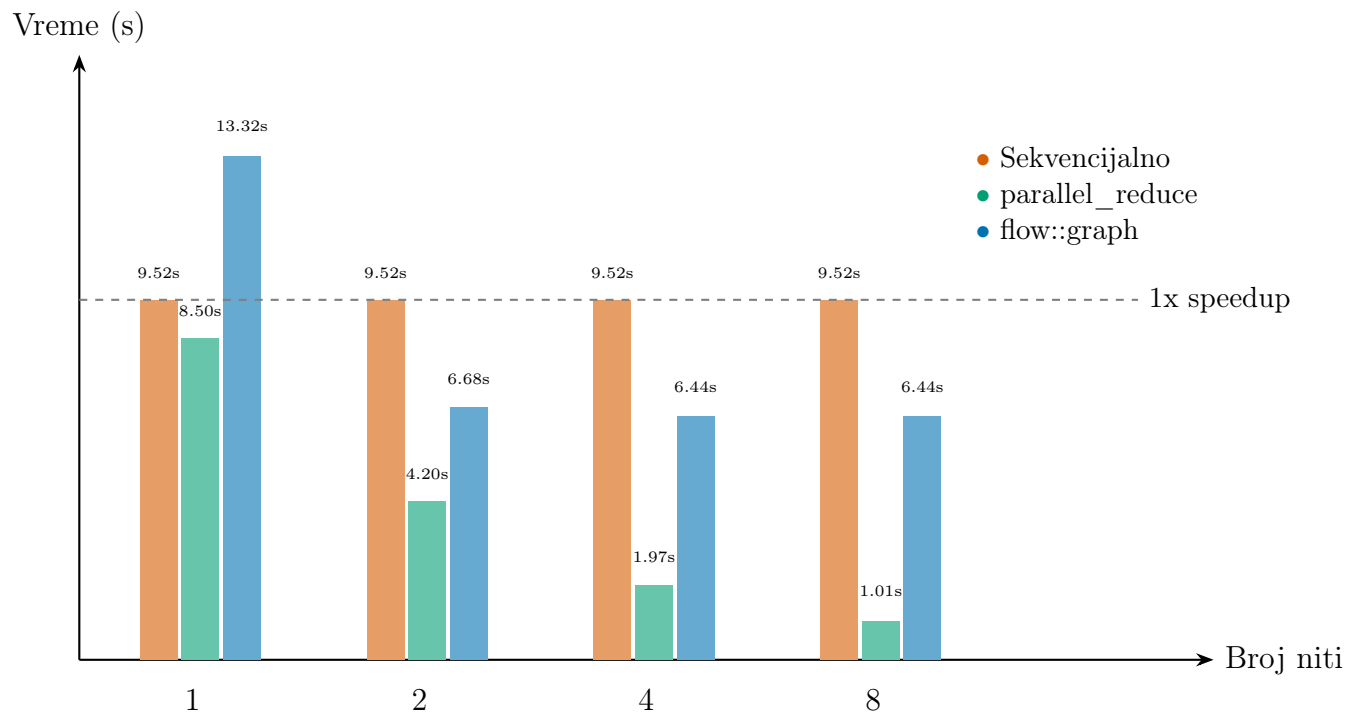
Listing 9: Primer izveštaja simulacije (1.000.000 spinova)

6.2 Benchmark rezultati

Benchmark je izvršen sa 1.000.000 spinova na sistemu sa 8 jezgara. Rezultati su prikazani u tabeli i na grafikonu.

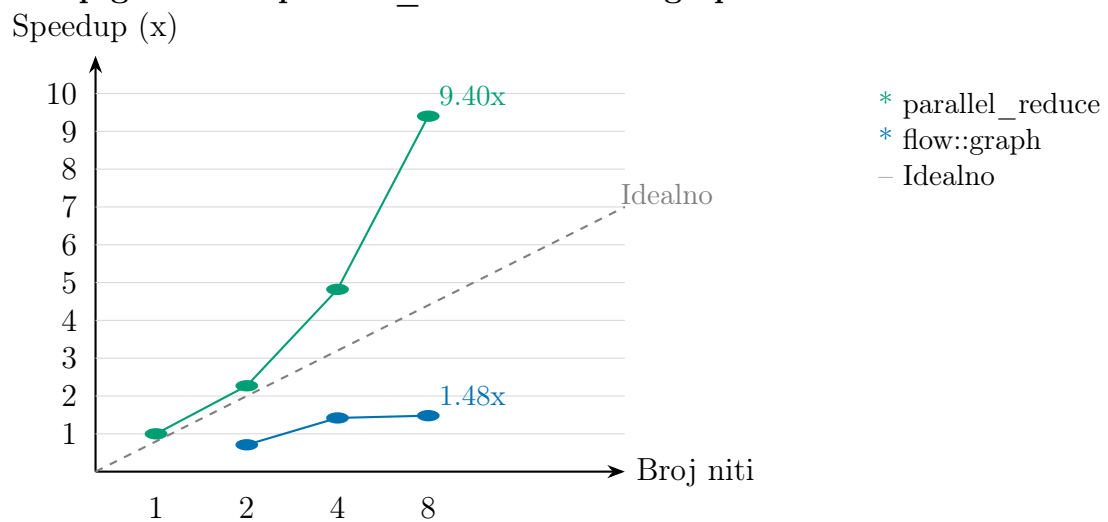
Tabela 4: Benchmark rezultati za 1.000.000 spinova

Metoda	Broj niti	Vreme (s)	Speedup
Sekvencijalno	1	9.52	1.00x
parallel_reduce	2	4.20	2.27x
flow::graph	2	13.32	0.71x
parallel_reduce	4	1.97	4.82x
flow::graph	4	6.68	1.42x
parallel_reduce	8	1.01	9.40x
flow::graph	8	6.44	1.48x



Slika 6: Uporedni prikaz vremena izvršenja

Speedup grafikon – parallel_reduce vs flow::graph



Slika 7: Speedup grafikon – parallel_reduce vs flow::graph

6.3 Analiza rezultata

Tabela 5: Analiza RTP komponenti

Komponenta	RTP	Učešće
Base game	80.91%	83.16%
Scatter	2.33%	2.39%
Bonus	14.04%	14.43%
Ukupno	97%	100%

Zapažanja:

- `parallel_reduce` pokazuje odličnu skalabilnost – speedup od 9.40x sa 8 niti
- `flow::graph` ima veći overhead zbog složenije strukture grafa (1.48x sa 8 niti)
- RTP od 97% je u okviru očekivanog opsega ($96.5\% \pm 0.5\%$)
- Hit frequency od 11.24% je očekivana za konfigurisanu slot igru
- Bonus RTP od 14.04% značajno doprinosi ukupnom RTP-u

7 Testiranje

7.1 Strategija testiranja

Testiranje je fokusirano na verifikaciju pojedinačnih komponenti i paralelnih algoritama. Testovi su implementirani koristeći Google Test framework.

7.2 Testni scenariji

Tabela 6: Testovi Monte Carlo simulatora

Test ID	Opis	Očekivani rezultat
T1	Sekvencijalna simulacija	Korektne statistike
T2	Paralelna simulacija (<code>parallel_reduce</code>)	Isti rezultati kao sekvencijalna
T3	Flow graph simulacija	Isti rezultati kao sekvencijalna
T4	RTP validacija	RTP u opsegu $96.5 \pm 0.5\%$
T5	Benchmark	Speedup $> 1x$ za više niti
T6	Statistika	Ispravni RTP, hit frequency, varijansa
T7	Flow graph struktura	Svi čvorovi korektno povezani

Tabela 7: Rezultati testiranja

Test	Status	Opis
Sekvencijalna simulacija	Prošao	Ispravne statistike
Paralelna simulacija	Prošao	Isti rezultati
Flow graph simulacija	Prošao	Isti rezultati
RTP validacija	Prošao	97% (unutar opsega)
Benchmark	Prošao	Speedup 9.40x

8 Zaključak

U okviru ovog projekta uspešno je implementiran paralelni Monte Carlo simulator za slot mašine. Projekat demonstrira:

- **Modularnu arhitekturu** – Slot Engine kao zasebna biblioteka (Git submodule), Monte Carlo simulator kao glavni projekat
- **Paralelizaciju** – Korišćenje Intel TBB za efikasnu simulaciju miliona spinova
- **Scalability** – Speedup od 9.40x sa 8 niti za `parallel_reduce`
- **Preciznost** – RTP od 97% (očekivano 96.5%) sa 95% intervalom poverenja [0.96, 0.99]

Ključni rezultati:

- `parallel_reduce`: 9.40x speedup sa 8 niti (1.01s za 1.000.000 spinova)
- `flow::graph`: 1.48x speedup sa 8 niti (6.44s za 1.000.000 spinova)
- RTP validiran na 97% (očekivano: 96.5%)
- Hit frequency: 11.24%
- Varijansa: 40.09, standardna devijacija: 6.33

Moguća unapređenja:

- Dodavanje više tipova bonus igara (pick-and-win, wheel of fortune)
- Podrška za progresivni džekpot
- GPU akceleracija (CUDA) za dodatno ubrzanje
- Optimizacija `flow::graph` za bolje performanse

Repozitorijum: <https://github.com/Dzekanaa/monte-carlo-simulator>

Datum: Juni 2026.

Literatura

1. Intel Corporation. *Intel Threading Building Blocks (TBB) Documentation*. 2025.
2. Stroustrup, B. *The C++ Programming Language*. 4th Edition. Addison-Wesley, 2013.
3. Williams, A. *C++ Concurrency in Action*. 2nd Edition. Manning, 2019.
4. Reinders, J. *Intel Threading Building Blocks*. O'Reilly, 2007.

Spisak slika

1	Konceptualni dijagram Monte Carlo simulacije	1
2	Arhitektura Monte Carlo simulatora	4
3	parallel_reduce model – paralelna simulacija spinova	7
4	enumerable_thread_specific – svaka nit ima sopstveni RNG i SlotEngine	7
5	Struktura flow::graph-a sa buffer node-ovima	8
6	Uporedni prikaz vremena izvršenja	12
7	Speedup grafikon – parallel_reduce vs flow::graph	12

Spisak tabela

1	Pregled funkcionalnih zahteva	2
2	Pregled programskih zahteva	2
3	Čvorovi flow::graph-a	9
4	Benchmark rezultati za 1.000.000 spinova	11
5	Analiza RTP komponenti	13
6	Testovi Monte Carlo simulatora	13
7	Rezultati testiranja	14